

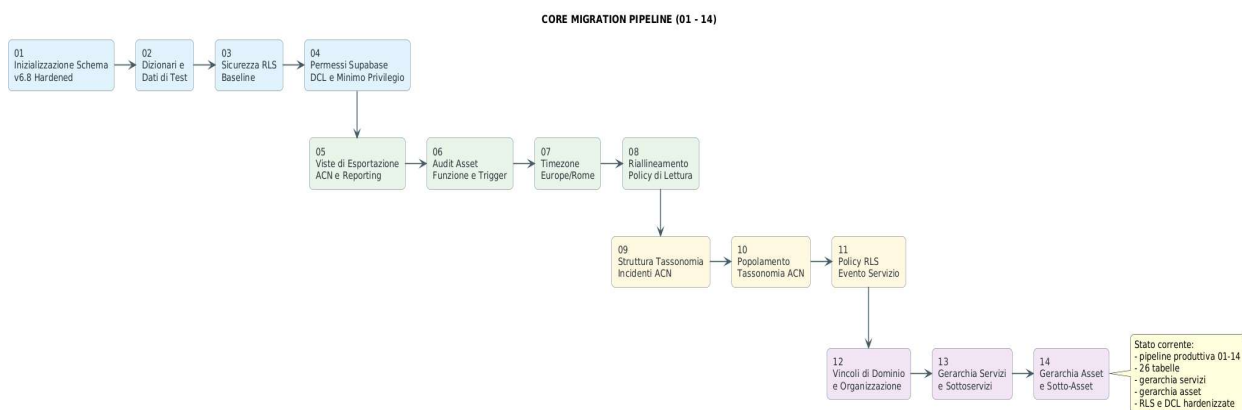
INGEGNERIZZAZIONE DEL BACKEND DATI: REPOSITORY DELLE MIGRAZIONI E STRUMENTI DI DIAGNOSTICA

La gestione del database relazionale, implementato su piattaforma PostgreSQL tramite l'istanza cloud Supabase, si basa su una struttura ordinata di script SQL storicizzati all'interno del repository Git di progetto. L'intera base di codice che governa la definizione dello schema, le politiche di sicurezza e il popolamento dei dati è organizzata in file sequenziali per garantirne la replicabilità e la tracciabilità.

Il codice è suddiviso in due rami logici complementari:

Core Migration Pipeline (01 - 14): Script sequenziali, ordinati e versionati, responsabili della creazione, del popolamento, dell'implementazione delle politiche di sicurezza e del refactoring controllato delle funzionalità di sistema.

Diagnostic & Patch Toolkit (X1 – X10): Un insieme di utilità adibite all'ispezione dei metadati, alla bonifica transizionale dei record simulati (data cleaning) e allo staging controllato dei componenti di audit trail in formato semistutturato.



1. LA CORE MIGRATION PIPELINE (01 - 14)

La sequenza numerata garantisce la riproducibilità deterministica dell'intero database partendo da un'istanza PostgreSQL vuota. Di seguito viene dettagliata l'analisi tecnica di ciascuno script.

01-Inizializzazione-Schema-v6.8_Hardened.sql

DDL & Schema Relazionale Core

Rappresenta il punto di ingresso dell'architettura. Contiene le istruzioni DDL (CREATE TABLE) per la generazione delle 22 tabelle della baseline relazionale. Le migrazioni successive introducono le due tabelle dedicate alla tassonomia degli incidenti ACN e, con lo script 14, le due tabelle della gerarchia asset-sotto-asset, portando il modello alle attuali 26 tabelle.

Gestione dell'Integrità: Lo script applica in modo ferreo i vincoli di integrità referenziale tramite chiavi primarie (PRIMARY KEY), esterne (FOREIGN KEY con clausole ON DELETE/UPDATE CASCADE o RESTRICT) e vincoli di dominio (CONSTRAINT ... CHECK).

La prima versione dell'Audit Engine: Viene qui dichiarata la prima stesura del meccanismo di tracciamento forense. Esso si compone di una tabella cumulativa pura (public.audit_log), di una stored function in linguaggio procedurale PL/pgSQL (public.process_asset_audit()) e di un trigger associato ad eventi DML (trg_asset_audit su public.asset). Questa prima implementazione esegue la serializzazione del record transazionale (OLD e NEW) tramite casting di riga a stringa di testo:

SQL

```
INSERT INTO public.audit_log (asset_id, utente, operazione,
valore_precedente, valore_nuovo)
VALUES (OLD.id, current_actor, 'UPDATE', ROW_TO_JSON(OLD)::text,
ROW_TO_JSON(NEW)::text);
```

02-dati.sql

Popolamento dei Dizionari e Record di Test

Questo script gestisce l'immissione del dataset simulato (DML).

Idempotenza della procedura: Per consentire l'esecuzione ciclica e ripetibile del seed senza incorrere in violazioni di univocità, lo script esegue preliminarmente un'istruzione TRUNCATE su 22 tabelle del dataset iniziale, utilizzando RESTART IDENTITY CASCADE per ripristinare le sequenze e includere le eventuali tabelle dipendenti attraverso i vincoli referenziali.

SQL

```
TRUNCATE public.audit_log, ..., public.organizzazione RESTART IDENTITY CASCADE;
```

Popolamento gerarchico: I dati vengono inseriti rispettando rigorosamente l'ordine imposto dai vincoli di chiave esterna, partendo dai dizionari e anagrafiche stabili (es. organizzazione, categoria_asset con la tassonomia ufficiale ACN come AC:NW_HW-FW per i firewall) fino ad arrivare alle tabelle transazionali di dipendenza e tracciamento incidentale.

03-sicurezza-rls.sql

Automazione delle Politiche di Row Level Security

Lo script introduce la configurazione di sicurezza Row Level Security per le tabelle appartenenti alla baseline del database.

Un blocco procedurale dinamico verifica l'esistenza delle tabelle, abilita la RLS e ricrea le policy applicative secondo un modello uniforme.

La policy Read_All_Policy consente la lettura:

- agli utenti autenticati con livello MFA aal2;
- all'utenza docente autorizzata docentepegaso@gmail.com.

Le policy Insert_MFA_Policy e Update_MFA_Policy vengono applicate esclusivamente alle tabelle operative e richiedono una sessione autenticata con livello MFA aal2.

Non viene definita alcuna policy DELETE. Le tabelle di dominio e la tabella audit_log rimangono disponibili in sola lettura per i ruoli applicativi.

04-Fix schema permissions for Supabase.sql

DCL & Modello Zero-Trust

Lo script configura i privilegi PostgreSQL assegnati ai ruoli applicativi, mantenendoli distinti dalle policy Row Level Security.

Il ruolo anon non dispone di privilegi sulle tabelle e sulle sequenze dello schema applicativo.

Il ruolo authenticated riceve:

- il privilegio SELECT sulle tabelle e sulle viste;
- i privilegi INSERT e UPDATE esclusivamente sulle tabelle operative.

Non viene concesso il privilegio DELETE.

Lo script configura inoltre i privilegi predefiniti per gli oggetti creati successivamente. Le nuove tabelle ricevono inizialmente il solo privilegio SELECT; gli eventuali permessi di scrittura devono essere concessi esplicitamente dalla relativa migrazione.

05_viste_esportazione_acn.sql

Reporting e Interoperabilità ACN/NIS2

Lo script crea gli oggetti logici (viste) per assolvere all'esportazione strutturata dei dati utili alla compliance NIS2. Vengono definite due viste chiave configurate con la clausola WITH (security_invoker = true) affinché la lettura rispetti i vincoli RLS del chiamante:

`public.vista_esportazione_acn_assets`: Esegue l'unione tra le tabelle degli asset fisici e logici, le categorie tassonomiche ACN e le vulnerabilità ad essi associati. Utilizza COALESCE per rimpiazzare i valori vuoti e standardizzare l'output per i parser di ACN (es. COALESCE (v.codice_bollettino, 'NESSUNA')).

`public.vista_reporting_servizi_critici`: Combina le dipendenze dei servizi verso gli asset tecnologici e i fornitori esterni, utilizzando le due relazioni molti-a-molti separate presenti nel modello. Nella versione corrente, quando uno stesso servizio possiede più asset e più fornitori, la combinazione delle join può produrre una moltiplicazione delle righe. La revisione della vista è rinviata al completamento del modello Supply Chain multilivello.

06_configurazione_audit_permissions.sql

Refactoring e Hardening dell'Audit Engine

Lo script consolida la funzione `fn_audit_asset_changes()` e il trigger `trg_asset_audit` associato alla tabella `asset`.

La funzione, dichiarata SECURITY DEFINER, registra nella tabella `audit_log`:

- il tipo di operazione eseguita;
- l'utente responsabile;
- l'identificativo dell'asset;
- la data e l'ora della modifica.

Quando l'operazione non è associata a un JWT applicativo, l'utente viene registrato come `SYSTEM_CORE`.

Il frontend non dispone del privilegio INSERT su `audit_log` e non può quindi scrivere direttamente nel registro. L'inserimento viene effettuato esclusivamente dalla funzione di audit eseguita con i privilegi del proprietario.

Le precedenti policy permissive vengono eliminate. La tabella `audit_log` resta disponibile in sola lettura per gli utenti autorizzati.

07_impostazione_timezone_locale.sql (Allineamento Temporale di Sistema)

La conformità e l'immutabilità dei log ai fini NIS2 richiedono un allineamento cronologico coerente e privo di discrepanze.

Configurazione del Timezone: Lo script configura il database PostgreSQL per utilizzare l'orario locale italiano (CET/CEST) impostando il fuso orario globale su 'Europe/Rome':

ALTER DATABASE postgres SET timezone TO 'Europe/Rome';

Questa configurazione imposta Europe/Rome come fuso orario predefinito delle nuove sessioni PostgreSQL, assicurando che le date e gli orari prodotti o visualizzati dal database siano coerenti con il contesto operativo italiano.

08-update-read-policy.sql

Riallineamento della policy di lettura

Lo script aggiorna la policy Read_All_Policy sulle tabelle della baseline, senza modificare le policy dedicate alle operazioni di inserimento e aggiornamento.

La lettura non è consentita genericamente a tutti gli utenti autenticati, ma esclusivamente:

- agli utenti con autenticazione MFA e livello aal2;
- all'utenza docente autorizzata docentepegaso@gmail.com.

Lo script elimina l'eventuale versione precedente della policy mediante DROP POLICY IF EXISTS e ricrea la regola di lettura in modo coerente con il modello di sicurezza definito nello script 03.

Le operazioni di scrittura restano separate e subordinate alle rispettive policy MFA.

09-tassonomia-incidenti-acn.sql

Integrazione della Tassonomia Cyber ACN

Lo script estende il modello relazionale introducendo le strutture necessarie alla classificazione degli incidenti secondo la Tassonomia Cyber pubblicata dall'Agenzia per la Cybersicurezza Nazionale. L'intervento aggiunge al database le entità dedicate alla memorizzazione delle categorie tassonomiche e alla loro associazione con gli eventi di servizio.

La tabella tassonomia_incidenti_acn conserva gli identificativi univoci, la macro-area, la sotto-categoria, il codice ACN, la denominazione estesa e la descrizione della voce tassonomica. La tabella associativa evento_tassonomia_acn collega ciascun evento alle classificazioni selezionate durante la procedura guidata di segnalazione.

Il modello risolve la relazione multi-a-molti tra eventi e voci tassonomiche mediante una chiave primaria composta. Il vincolo CHECK applicato a passo_wizard ammette valori compresi tra 1 e 6, in coerenza con la sequenza implementata nell'interfaccia web.

10-popolamento-tassonomia-acn.sql

Popolamento controllato della Tassonomia Cyber ACN

Lo script completa l'integrazione strutturale introdotta dalla migrazione precedente, popolando la tabella tassonomia_incidenti_acn con le voci necessarie alla classificazione degli eventi cyber.

Per ciascuna voce vengono memorizzati il codice ACN, la macro-area, la sotto-categoria, la denominazione estesa e gli ulteriori metadati descrittivi utilizzati dall'applicazione.

Il popolamento utilizza la clausola:

ON CONFLICT (codice_acn) DO NOTHING

Questa istruzione impedisce la duplicazione delle voci già presenti e permette di rieseguire il popolamento senza violare il vincolo di univocità applicato a codice_acn.

La separazione tra lo script 09, responsabile della creazione delle strutture relazionali, e lo script 10, responsabile del popolamento dei dati, mantiene distinte la definizione dello schema DDL e la gestione del dataset tassonomico DML.

11-policy-evento-servizio.sql

Hardening RLS degli eventi di servizio

Lo script configura in modo specifico la sicurezza della tabella `evento_servizio`, riallineandola al modello generale di autenticazione adottato dal progetto.

Le precedenti policy permissive basate su `USING (true)` e `WITH CHECK (true)` vengono eliminate e sostituite dalle policy canoniche:

- `Read_All_Policy;`
- `Insert_MFA_Policy;`
- `Update_MFA_Policy.`

La lettura è consentita agli utenti che dispongono di una sessione MFA con livello `aal2` oppure all'utenza docente autorizzata `docentepegaso@gmail.com`.

Le operazioni di inserimento e aggiornamento richiedono invece una sessione MFA `aal2`. Non viene definita alcuna policy di cancellazione e il privilegio PostgreSQL `DELETE` non viene assegnato al ruolo `authenticated`.

Lo script riallinea inoltre i privilegi DCL della tabella, concedendo esclusivamente:

- `SELECT;`
- `INSERT;`
- `UPDATE.`

La verifica successiva ha confermato l'assenza delle policy legacy `auth_insert_evento_servizio` e `auth_select_evento_servizio`, nonché l'assenza del privilegio di cancellazione.

12-Hardening-Vincoli-Dominio-Organizzazione-v1.0.sql

Rafforzamento dei vincoli di dominio e della responsabilità organizzativa

Lo script applica un intervento di hardening successivo alla verifica diagnostica effettuata mediante `X8_CHECK_DUPLICATES_AND_NULL_ORGANIZATIONS.sql`.

L'operazione ha previsto innanzitutto l'associazione dei tre servizi privi di organizzazione all'entità organizzativa principale del dataset, identificata come "Azienda Sanitaria Territoriale Alpha". Dopo la bonifica, sia gli asset sia i servizi risultano obbligatoriamente collegati a un'organizzazione.

Sono stati quindi introdotti vincoli `UNIQUE` sui seguenti attributi:

- `tipo_servizio.nome;`
- `tipo_fornitore.nome;`
- `ruolo_organigramma.nome.`

Questi vincoli impediscono la duplicazione delle voci utilizzate come domini applicativi. Le colonne `asset.organizzazione_id` e `servizio.organizzazione_id` sono state rese `NOT NULL`, rafforzando il requisito di attribuzione organizzativa previsto dal modello.

Lo script opera all'interno di una transazione atomica. In presenza di anomalie o dell'assenza dell'organizzazione di riferimento, l'intera operazione viene annullata senza lasciare il database in uno stato parzialmente modificato.

13-Hardening-Gerarchia-Servizi-v1.0.sql

Hardening della gerarchia servizio-sottoservizio

Lo script introduce un codice applicativo stabile per ciascun servizio e trasforma servizio_componente in una relazione gerarchica storicizzabile. La tabella utilizza ora una chiave primaria tecnica autonoma e conserva i metadati di validità temporale, lo stato della relazione, l'eventuale padre primario e l'ordine di visualizzazione dei sottoservizi. Vincoli, indici parziali e un trigger PL/pgSQL impediscono autorelazioni, duplicati attivi, padri primari multipli e cicli nella gerarchia. Le foreign key verso servizio utilizzano ON DELETE RESTRICT, evitando la cancellazione automatica delle relazioni storiche.

14-Hardening-Gerarchia-Asset-v1.0.sql

Hardening della gerarchia asset-sotto-asset

Lo script introduce una struttura gerarchica multilivello per rappresentare le relazioni tra asset principali e sotto-asset.

La tabella asset viene estesa con il campo codice_asset, utilizzato come identificativo applicativo stabile e indipendente dalla chiave numerica interna. Per gli asset già esistenti, il codice viene generato a partire dal nome normalizzato oppure, nei casi non conformi o duplicati, mediante un valore legacy basato sull'identificativo del record.

La colonna asset.categoria_asset_id viene resa obbligatoria, assicurando che ogni asset sia associato a una categoria.

Lo script crea inoltre due nuove tabelle:

- tipo_relazione_asset, contenente le tipologie normalizzate di relazione;
- asset_componente, destinata a rappresentare il collegamento tra asset padre e asset figlio.

La relazione conserva lo stato di attività, il periodo di validità, l'eventuale indicazione del padre primario e l'ordine di visualizzazione dei componenti.

Vincoli, indici parziali e un trigger PL/pgSQL impediscono:

- l'associazione di un asset con sé stesso;
- la duplicazione di relazioni attive;
- la presenza di più padri primari attivi per lo stesso sotto-asset;
- la riattivazione di relazioni storiche chiuse;
- la creazione di cicli gerarchici del tipo $A \rightarrow B \rightarrow C \rightarrow A$.

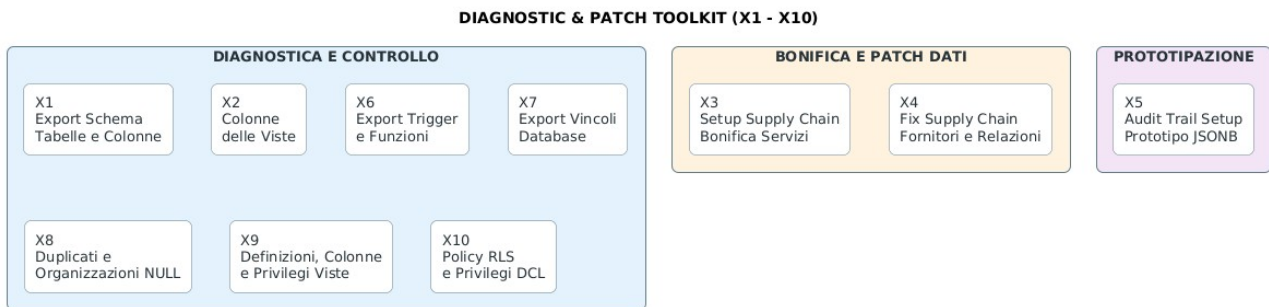
Le foreign key utilizzano ON DELETE RESTRICT, evitando la cancellazione automatica degli asset collegati.

Le nuove tabelle sono protette mediante Row Level Security. La lettura è consentita alle sessioni MFA con livello aal2 oppure all'utenza docente autorizzata, mentre il ruolo anon non dispone di privilegi.

Con questa migrazione il modello raggiunge complessivamente **26 tabelle** e comprende sia la gerarchia servizio-sottoservizio sia la gerarchia asset-sotto-asset.

2. IL DIAGNOSTIC & PATCH TOOLKIT (X1 – X10)

La serie dei file identificati dal prefisso X raccoglie tutti gli script necessari alle attività di diagnostica del database live e alla correzione "a caldo" (hotfix) delle anomalie relazionali individuate durante i cicli di test.



X1_EXPORT_SCHEMA_db.sql (Estrazione Metadati dello Schema)

Questo script opera direttamente sul catalogo di sistema del database (information_schema).

- **Analisi strutturale:** Interroga l'unione sistemica tra le viste tables e columns filtrate sullo schema public. Estrae il nome di ogni tabella, la lista ordinata dei relativi attributi, il tipo di dato e la nullabilità:

SQL

```
SELECT t.table_name, c.column_name, c.data_type, c.is_nullable
FROM information_schema.tables t
JOIN information_schema.columns c ON t.table_name = c.table_name
WHERE t.table_schema = 'public'
ORDER BY t.table_name, c.ordinal_position;
```

- **Utilità nel progetto:** Questo script fornisce l'output grezzo strutturato con cui è stato generato il *Data Dictionary* ufficiale del Project Work, consentendo di ispezionare istantaneamente lo schema fisico senza utilizzare interfacce grafiche.

X2_VISUALIZZA_COLONNE_VISTE.sql (Diagnostica dell'Interoperabilità)

Esegue un'interrogazione mirata a livello strutturale del dizionario di PostgreSQL ma focalizzata esclusivamente sulle viste create nello script 05.

- **Validazione formale:** Consente di analizzare i campi esposti e i relativi tipi generati da vista_esportazione_acn_assets e vista_reporting_servizi_critici. È uno strumento di controllo di qualità che permette di verificare l'esatto tipo di dato derivante dalle join complesse prima di esporre la risorsa all'interfaccia client o all'esportatore CSV.

X3_SETUP_SUPPLY_CHAIN.sql (DML Patch - Bonifica delle Tipologie di Servizio)

Si tratta di un intervento di allineamento e bonifica dei record (Data Cleaning) sulla tabella dei servizi.

- **L'uso strategico di LIMIT 1:** Durante i test di popolamento dei servizi critici, per prevenire errori legati alla presenza accidentale di omonimi nella tabella di lookup tipo_servizio, lo script esegue query condizionali blindate:

SQL

```
UPDATE servizio
SET tipo_servizio_id = (SELECT id FROM tipo_servizio WHERE nome = 'Infrastruttura'
LIMIT 1)
WHERE nome IN ('Gestione Identità (Active Directory)', 'Rete e Connettività Perimetrale');
```

L'utilizzo di LIMIT 1 impedisce al motore di database di sollevare l'eccezione SQL 21000 (cardinalità della sottoquery superiore a uno), un'ottima pratica per garantire l'idempotenza e la robustezza delle query di update condizionali basate su sub-select.

X4_FIX_SUPPLY_CHAIN_FW.sql (DML Patch - Chiusura della Catena di Dipendenze)

Questo script completa l'allineamento della catena relazionale introducendo elementi reali della Supply Chain.

- **Fornitura NGFW:** Inserisce il fornitore "Palo Alto Networks" all'interno dell'anagrafica e successivamente chiude la relazione multi-a-molti con il servizio core del database simulato ("Pronto Soccorso Real-Time"):

SQL

```
INSERT INTO servizio_dipendenza_fornitore (servizio_id, fornitore_id, descrizione)
SELECT s.id, f.id, 'Fornitura NGFW e supporto H24'
FROM servizio s, fornitore f
WHERE s.nome = 'Pronto Soccorso Real-Time' AND f.nome = 'Palo Alto Networks';
```

Questo script consolida il dataset di test, dimostrando l'efficacia del modello nel tracciare i vendor critici per i servizi di emergenza in un contesto ospedaliero.

X5_AUDIT_TRAIL_SETUP.sql

Questo script implementa il codice prototipale per l'evoluzione del sistema di audit in formato semistrutturato (JSONB).

Casting Semistrutturato in PL/pgSQL: Diversamente dalla versione originaria del file 01, questa funzione introduce l'uso di to_jsonb(RECORD) per catturare l'intero record modificato a livello transazionale, salvandone lo stato all'interno dei campi testuali (valore_precedente e valore_nuovo) dell'audit log:

SQL

```
IF (TG_OP = 'UPDATE') THEN
    INSERT INTO audit_log (asset_id, utente, operazione, data_modifica, valore_precedente,
valore_nuovo)
    VALUES (NEW.id, current_user, 'UPDATE', now(), to_jsonb(OLD)::text,
to_jsonb(NEW)::text);
    RETURN NEW;
...
```


Vantaggio Architettuale: L'approccio basato sulla serializzazione JSON consente di acquisire dinamicamente lo stato del record, riducendo la necessità di modificare la funzione di audit quando la struttura delle tabelle viene estesa con nuovi attributi. La riga viene serializzata dinamicamente in una stringa JSON valida, rendendo l'infrastruttura di auditing altamente scalabile ed estensibile.

X6_EXPORT_TRIGGER_FUNCTIONS.sql

Esportazione dei trigger e delle funzioni associate

Lo script diagnostico interroga i cataloghi interni di PostgreSQL per estrarre i trigger non interni presenti nel database, le tabelle sulle quali operano e la definizione completa delle funzioni richiamate.

L'esecuzione ha permesso di identificare come componente applicativo del progetto il trigger:

public.trg_asset_audit associato alla tabella public.asset e alla funzione:

public.fn_audit_asset_changes()

Il trigger si attiva dopo le operazioni di INSERT, UPDATE e DELETE e registra nella tabella audit_log il tipo di operazione, l'utente, l'identificativo dell'asset e la data della modifica.

Gli altri trigger individuati negli schemi storage e realtime appartengono all'infrastruttura interna di Supabase e non costituiscono componenti sviluppati specificamente per il Project Work.

X7_EXPORT_DATABASE_CONSTRAINTS.sql

Verifica delle chiavi e dei vincoli del modello relazionale

Lo script interroga i cataloghi PostgreSQL per estrarre le chiavi primarie, le chiavi esterne, i vincoli UNIQUE e i vincoli CHECK presenti nello schema public.

La verifica ha confermato:

- la presenza di una chiave primaria per ciascuna tabella;
- l'impiego di chiavi primarie composte nelle tabelle associative;
- la presenza delle relazioni di integrità referenziale;
- il controllo dei domini relativi a criticità, severità, remediation e operazioni di audit;
- il vincolo del wizard compreso tra i passi 1 e 6;
- l'univocità dei principali codici e identificatori ACN.

Lo script costituisce una delle evidenze tecniche utilizzate per verificare la coerenza strutturale del modello e supportare l'analisi del processo di normalizzazione relazionale.

X8_CHECK_DUPLICATES_AND_NULL_ORGANIZATIONS.sql

Controllo preventivo di duplicati e associazioni organizzative mancanti

Lo script esegue controlli di sola lettura prima dell'introduzione di nuovi vincoli strutturali.

Le verifiche hanno riguardato:

- duplicati in tipo_servizio.nome;
- duplicati in tipo_fornitore.nome;
- duplicati in ruolo_organigramma.nome;
- asset privi di organizzazione;
- servizi privi di organizzazione.

Non sono stati rilevati duplicati nelle tre tabelle di dominio e tutti gli asset risultavano già associati a un'organizzazione. Sono invece stati individuati tre servizi con organizzazione_id nullo:

- Gestione Identità, basata su Active Directory;
- Rete e Connettività Perimetrale;
- Database Cartelle Cliniche, EMR.

Il risultato ha costituito il presupposto per l'esecuzione controllata dello script produttivo 12, che ha bonificato i dati e introdotto i nuovi vincoli.

Stato reale del trigger di audit in produzione

La verifica diretta eseguita sul catalogo PostgreSQL ha confermato che il trigger attivo sulla tabella asset è trg_asset_audit, collegato alla funzione fn_audit_asset_changes().

La funzione attualmente in produzione registra:

- l'operazione DML eseguita;
- l'indirizzo e-mail dell'utente autenticato, quando disponibile;
- l'identificativo dell'asset interessato;
- la data e l'ora dell'evento.

La struttura della tabella audit_log dispone anche dei campi valore_precedente e valore_nuovo; tuttavia, la funzione attualmente attiva non li valorizza. Le versioni contenute nello script iniziale e nel prototipo X5 documentano l'evoluzione progettuale verso la serializzazione del record, ma non devono essere descritte come funzionalità attualmente operative.

La scelta di non applicare una chiave esterna tra audit_log.asset_id e asset.id è intenzionale. Essa consente di conservare l'evidenza storica dell'operazione anche quando l'asset originario viene eliminato. In questo caso la persistenza del registro di audit è considerata prioritaria rispetto al vincolo referenziale diretto.

X9_EXPORT_VIEWS_DEFINITIONS_AND_PRIVILEGES.sql

Verifica delle viste applicative e dei privilegi

Lo script diagnostico esporta l'esistenza, la definizione SQL, le colonne, il proprietario, i privilegi e la configurazione security_invoker delle viste vista_esportazione_acn_assets e vista_reporting_servizi_critici. La verifica ha confermato che entrambe le viste esistono, appartengono allo schema public e utilizzano security_invoker. L'analisi ha inoltre evidenziato privilegi DML non necessari e una moltiplicazione delle righe nella vista Supply Chain quando uno stesso servizio possiede più asset e più fornitori. La correzione della vista sarà effettuata dopo il completamento del modello Supply Chain multilivello.

X10_EXPORT_RLS_POLICIES_AND_GRANTS.sql

Esportazione delle policy RLS e dei privilegi applicativi

Lo script diagnostico estrae lo stato delle policy Row Level Security e dei privilegi assegnati alle tabelle dello schema public.

Per ciascuna tabella vengono verificati:

- attivazione della RLS;
- policy presenti e relativo comando;
- ruoli autorizzati;

- condizioni `USING` e `WITH CHECK`;
- privilegi concessi ai ruoli `anon` e `authenticated`;
- eventuale presenza dell'indirizzo docente obsoleto;
- presenza dell'indirizzo corretto `docentepegaso@gmail.com`.

L'esecuzione dello script ha confermato l'assenza di privilegi per il ruolo `anon`, l'assenza del privilegio `DELETE` per il ruolo `authenticated` e il corretto allineamento delle policy al modello MFA/docente.

3. ANALISI DELLE RIDONDANZE E REFACTORING CONTROLLATO

Nel contesto di uno sviluppo agile, la sovrapposizione temporale di alcune istruzioni all'interno del repository non rappresenta un difetto di programmazione, bensì la **documentazione storica del refactoring applicativo**.

Nel database implementato si evidenziano tre scenari significativi di sovrapposizione/refactoring:

A. Sovrascrittura della Logica di Audit Log (File 01 vs X5 vs 06)

Il tracciamento delle modifiche transazionali subisce un'evoluzione in tre fasi distinte:

Fase Iniziale (01): La funzione scrive i dati nel log convertendo l'intera riga tramite `ROW_TO_JSON(NEW)::text`. Per ricavare l'attore dell'operazione, la funzione interroga direttamente i metadati del JWT di Supabase tramite `auth.jwt() ->> 'email'`.

Fase Prototipale di Debug (X5): Viene testata la variante `to_jsonb()` per ottimizzare la serializzazione e viene provato il ruolo di sessione standard PostgreSQL `current_user`.

Fase Consolidata e Hardened (06): La funzione definitiva viene ottimizzata e consolidata utilizzando il trigger `trg_asset_audit` su `asset` con la stored function `fn_audit_asset_changes()`. Si utilizza la funzione nativa di Supabase `auth.email()` inserendo l'operazione transazionale e delegando le informazioni dettagliate dei record ad altre sorgenti, semplificando la struttura del log di sicurezza ai fini della performance.

B. Duplicazione delle Istruzioni di Abilitazione RLS (File 03 vs 06)

Nel file 03-sicurezza-rls.sql viene abilitata la sicurezza a livello di riga sulle 24 tabelle della baseline, incluse `asset` e `audit_log`. Le tabelle introdotte dalle migrazioni successive ricevono la propria configurazione RLS all'interno dei rispettivi script produttivi.

Nel file 06_configurazione_audit_permissions.sql viene reiterato esplicitamente il comando `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` per la tabella `audit_log`.

Valutazione Tecnica: In PostgreSQL, l'abilitazione multipla della RLS su una tabella che l'ha già attiva è un'operazione sicura e innocua (operazione idempotente). Il motore di database ignora il comando secondario senza sollevare eccezioni o alterare lo stato dei permessi correnti.

C. Riallineamento delle Policy di Accesso in Lettura

File 03 e file 08

Lo script 03 definisce la configurazione iniziale delle policy RLS, consentendo la lettura agli utenti autenticati con livello MFA `aa12` oppure all'utenza docente autorizzata

`docentepegaso@gmail.com`.

Una versione intermedia dello script 08 aveva sostituito questa regola con una policy aperta a tutti gli utenti autenticati mediante la condizione `USING (true)`.

Durante il successivo hardening, lo script 08 è stato riallineato al modello definito dallo script 03. La versione corrente ricrea la policy `Read_All_Policy` utilizzando nuovamente la condizione MFA/docente.

L'uso di `DROP POLICY IF EXISTS` seguito dalla ricreazione della policy garantisce una sostituzione controllata della regola di lettura, evitando policy duplicate o conflittuali